

Technology Stack

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution.

S.NO	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, CSS3 (custom, no framework), Vanilla JavaScript	Keeps the app dependency-light; CSS variables power the theme switch and the animated SVG risk gauge without needing a JS framework.
2	Backend / Server-Side	Python, Flask	Lightweight framework that's a natural fit for wrapping a scikit-learn model behind a small number of routes.
3	Machine Learning	Scikit-learn (Decision Tree, Random Forest, KNN, Gradient Boosting compared)	Standard, well-documented classifiers suited to a small structured tabular dataset; easy to compare and pick the best performer.
4	Model Serialization	Joblib (floods.save, transform.save)	Efficient serialization for scikit-learn objects; keeps the trained model and its fitted scaler decoupled from

			the training notebook.
5	Data Handling	Pandas, NumPy	Used to assemble the incoming form values into the exact DataFrame shape/column names the scaler and model expect.
6	Production Server	Gunicorn	Flask's built-in dev server isn't meant for production; Gunicorn is the standard WSGI server for real traffic.
7	Cloud / Hosting / Deployment	Render	Simple Git-based deploys with a Procfile, free tier suitable for a portfolio project.
8	Version Control	Git, GitHub	Source control and the delivery mechanism for Render's build pipeline.
9	API Testing	Postman, curl	Used to test the /predict endpoint directly with form-encoded payloads, independent of the browser UI.